

Module 2

Lab - Lists and Tuples

Tuples in Python

Code:

```
tuple1 = ("disco",10,1.2 )  
tuple1
```

Output:

```
('disco', 10, 1.2)
```

Code:

```
type(tuple1)
```

Output:

```
<class 'tuple'>
```

Indexing

Code:

```
print(tuple1[0])  
print(tuple1[1])  
print(tuple1[2])
```

Output:

```
disco  
10  
1.2
```

Code:

```
print(type(tuple1[0]))  
print(type(tuple1[1]))  
print(type(tuple1[2]))
```

Output:

```
<class 'str'>  
<class 'int'>  
<class 'float'>
```

Code:

```
tuple1[-1]
```

Output:

```
1.2
```

Code:

```
tuple1[-2]
```

Output:

```
10
```

Code:
tuple1[-3]

Output:
'disco'

Concatenate Tuples

Code:
tuple2 = tuple1 + ("hard rock", 10)
tuple2

Output:
('disco', 10, 1.2, 'hard rock', 10)

Slicing

Code:
tuple2[0:3]

Output:
('disco', 10, 1.2)

Code:
tuple2[3:5]

Output:
('hard rock', 10)

Code:
len(tuple2)

Output:
5

Sorting

Code:
Ratings = (0, 9, 6, 5, 10, 8, 9, 6, 2)

Output:
no output, this line just stores the value

Code:
RatingsSorted = sorted(Ratings)
RatingsSorted

Output:
[0, 2, 5, 6, 6, 8, 9, 9, 10]

Nested Tuple

Code:

```
NestedT = (1, 2, ("pop", "rock") , (3,4), ("disco", (1,2)))
```

Output:

no output, this line just stores the value

Code:

```
print("Element 0 of Tuple: ", NestedT[0])
print("Element 1 of Tuple: ", NestedT[1])
print("Element 2 of Tuple: ", NestedT[2])
print("Element 3 of Tuple: ", NestedT[3])
print("Element 4 of Tuple: ", NestedT[4])
```

Output:

```
Element 0 of Tuple: 1
Element 1 of Tuple: 2
Element 2 of Tuple: ('pop', 'rock')
Element 3 of Tuple: (3, 4)
Element 4 of Tuple: ('disco', (1, 2))
```

Code:

```
print("Element 2, 0 of Tuple: ", NestedT[2][0])
print("Element 2, 1 of Tuple: ", NestedT[2][1])
print("Element 3, 0 of Tuple: ", NestedT[3][0])
print("Element 3, 1 of Tuple: ", NestedT[3][1])
print("Element 4, 0 of Tuple: ", NestedT[4][0])
print("Element 4, 1 of Tuple: ", NestedT[4][1])
```

Output:

```
Element 2, 0 of Tuple: pop
Element 2, 1 of Tuple: rock
Element 3, 0 of Tuple: 3
Element 3, 1 of Tuple: 4
Element 4, 0 of Tuple: disco
Element 4, 1 of Tuple: (1, 2)
```

Code:

```
NestedT[2][1][0]
```

Output:

```
'r'
```

Code:

```
NestedT[2][1][1]
```

Output:

```
'o'
```

Code:

```
NestedT[4][1][0]
```

Output:

```
1
```

Code:

```
NestedT[4][1][1]
```

Output:

```
2
```

Quiz on Tuples

Code:

```
genres_tuple = ("pop", "rock", "soul", "hard rock", "soft rock", \
                "R&B", "progressive rock", "disco")
genres_tuple
```

Output:

```
('pop', 'rock', 'soul', 'hard rock', 'soft rock', 'R&B', 'progressive rock',
'disco')
```

Code:

```
len(genres_tuple)
```

Output:

```
8
```

Code:

```
genres_tuple[3]
```

Output:

```
'hard rock'
```

Code:

```
genres_tuple[3:6]
```

Output:

```
('hard rock', 'soft rock', 'R&B')
```

Code:

```
genres_tuple[0:2]
```

Output:

```
('pop', 'rock')
```

Code:

```
"disco".find('s')
```

Output:

```
2
```

Code:

```
C_tuple = (-5, 1, -3)
C_list = sorted(C_tuple)
C_list
```

Output:

```
[-5, -3, 1]
```

Lab completed - Tuples in Python.

Lists in Python

Indexing

Code:

```
L = ["The Bodyguard", 7.0, 1992]  
L
```

Output:

```
['The Bodyguard', 7.0, 1992]
```

Code:

```
print('the same element using negative and positive indexing:\n  
Postive:',L[0],  
'\n Negative:' , L[-3] )  
print('the same element using negative and positive indexing:\n  
Postive:',L[1],  
'\n Negative:' , L[-2] )  
print('the same element using negative and positive indexing:\n  
Postive:',L[2],  
'\n Negative:' , L[-1] )
```

Output:

```
the same element using negative and positive indexing:  
Postive: The Bodyguard  
Negative: The Bodyguard  
the same element using negative and positive indexing:  
Postive: 7.0  
Negative: 7.0  
the same element using negative and positive indexing:  
Postive: 1992  
Negative: 1992
```

List Content

Code:

```
["The Bodyguard", 7.0, 1992, [1, 2], ("A", 1)]
```

Output:

```
['The Bodyguard', 7.0, 1992, [1, 2], ('A', 1)]
```

List Operations

Code:

```
L = ["The Bodyguard", 7.0,1992,"BG",1]  
L
```

Output:

```
['The Bodyguard', 7.0, 1992, 'BG', 1]
```

Code:

```
L[3:5]
```

Output:

```
['BG', 1]
```

Code:

```
L = [ "The Bodyguard", 7.0]
L.extend(['pop', 10])
L
```

Output:

```
['The Bodyguard', 7.0, 'pop', 10]
```

Code:

```
L = [ "The Bodyguard", 7.0]
L.append(['pop', 10])
L
```

Output:

```
['The Bodyguard', 7.0, ['pop', 10]]
```

Code:

```
L.append(['a', 'b'])
L
```

Output:

```
['The Bodyguard', 7.0, ['pop', 10], ['a', 'b']]
```

Code:

```
A = ["disco", 10, 1.2]
print('Before change:', A)
A[0] = 'hard rock'
print('After change:', A)
```

Output:

```
Before change: ['disco', 10, 1.2]
After change: ['hard rock', 10, 1.2]
```

Code:

```
print('Before change:', A)
del(A[0])
print('After change:', A)
```

Output:

```
Before change: ['hard rock', 10, 1.2]
After change: [10, 1.2]
```

Code:

```
'hard rock'.split()
```

Output:

```
['hard', 'rock']
```

Code:

```
'A,B,C,D'.split(',')
```

Output:

```
['A', 'B', 'C', 'D']
```

Copy and Clone List**Code:**

```
A = ["hard rock", 10, 1.2]  
B = A  
print('A:', A)  
print('B:', B)
```

Output:

```
A: ['hard rock', 10, 1.2]  
B: ['hard rock', 10, 1.2]
```

Code:

```
print('B[0]:', B[0])  
A[0] = "banana"  
print('B[0]:', B[0])
```

Output:

```
B[0]: hard rock  
B[0]: banana
```

Code:

```
B = A[:]  
B
```

Output:

```
['banana', 10, 1.2]
```

Code:

```
print('B[0]:', B[0])  
A[0] = "hard rock"  
print('B[0]:', B[0])
```

Output:

```
B[0]: banana  
B[0]: banana
```

Quiz on List**Code:**

```
a_list = [1, 'hello', [1,2,3] , True]  
a_list
```

Output:

```
[1, 'hello', [1, 2, 3], True]
```

Code:

```
a_list[1]
```

Output:

```
'hello'
```

Code:

```
a_list[1:4]
```

Output:

```
['hello', [1, 2, 3], True]
```

Code:

```
A = [1, 'a']
```

```
B = [2, 1, 'd']
```

```
A + B
```

Output:

```
[1, 'a', 2, 1, 'd']
```

Scenario: Shopping List

Task-1 Create an empty list

Code:

```
Shopping_list = []
```

```
Shopping_list
```

Output:

```
[]
```

Task-2 Now store the number of items to the shopping_list

Code:

```
Shopping_list = ["Watch", "Laptop", "Shoes", "Pen", "Clothes"]
```

```
Shopping_list
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Pen', 'Clothes']
```

Task-3 Add a new item to the shopping_list

Code:

```
Shopping_list.append("Football")
```

```
Shopping_list
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Pen', 'Clothes', 'Football']
```

Task-4 Print First item from the shopping_list

Code:


```
print(Shopping_list[0])
```

Output:

Watch

Task-5 Print Last item from the shopping_list

Code:

```
print(Shopping_list[-1])
```

Output:

Football

Task-6 Print the entire Shopping List

Code:

```
print(Shopping_list)
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Pen', 'Clothes', 'Football']
```

Task-7 Print the items that are important to buy from the Shopping List

Code:

```
print(Shopping_list[1:3])
```

Output:

```
['Laptop', 'Shoes']
```

Task-8 Change the item from the shopping_list

Code:

```
Shopping_list[3] = "Notebook"
```

```
Shopping_list
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Notebook', 'Clothes', 'Football']
```

Task-9 Delete the item from the shopping_list that is not required

Code:

```
del(Shopping_list[4])
```

```
Shopping_list
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Notebook', 'Football']
```

Task-10 Print the shopping list

Code:

```
print(Shopping_list)
```

Output:

```
['Watch', 'Laptop', 'Shoes', 'Notebook', 'Football']
```

Lab completed - Lists in Python.

Sets

Code:

```
set1 = {"pop", "rock", "soul", "hard rock", "rock", "R&B", "rock", "disco"}  
set1
```

output:

```
{'R&B', 'disco', 'hard rock', 'pop', 'rock', 'soul'}
```

Code:

```
album_list = [ "Michael Jackson", "Thriller", 1982, "00:42:19", \  
               "Pop, Rock, R&B", 46.0, 65, "30-Nov-82", None, 10.0]
```

```
album_set = set(album_list)
```

```
album_set
```

output:

```
{'00:42:19',  
 10.0,  
 1982,  
 '30-Nov-82',  
 46.0,  
 65,  
 'Michael Jackson',  
 None,  
 'Pop, Rock, R&B',  
 'Thriller'}
```

Code:

```
music_genres = set(["pop", "pop", "rock", "folk rock", "hard rock", "soul", \  
                   "progressive rock", "soft rock", "R&B", "disco"])
```

```
music_genres
```

output:

```
{'R&B',  
 'disco',  
 'folk rock',  
 'hard rock',  
 'pop',  
 'progressive rock',  
 'rock',  
 'soft rock',  
 'soul'}
```

Code:

```
A = set(["Thriller", "Back in Black", "AC/DC"])
```

```
A
```

Output:

```
{'AC/DC', 'Back in Black', 'Thriller'}
```

Code:

```
A.add("NSYNC")
```

```
A
```

Output:

```
{'AC/DC', 'Back in Black', 'NSYNC', 'Thriller'}
```

Code:

```
A.add("NSYNC")
```

```
A
```

Output:

```
{'AC/DC', 'Back in Black', 'NSYNC', 'Thriller'}
```

Code:

```
A.remove("NSYNC")
```

```
A
```

Output:

```
{'AC/DC', 'Back in Black', 'Thriller'}
```

Code:

```
"AC/DC" in A
```

Output:

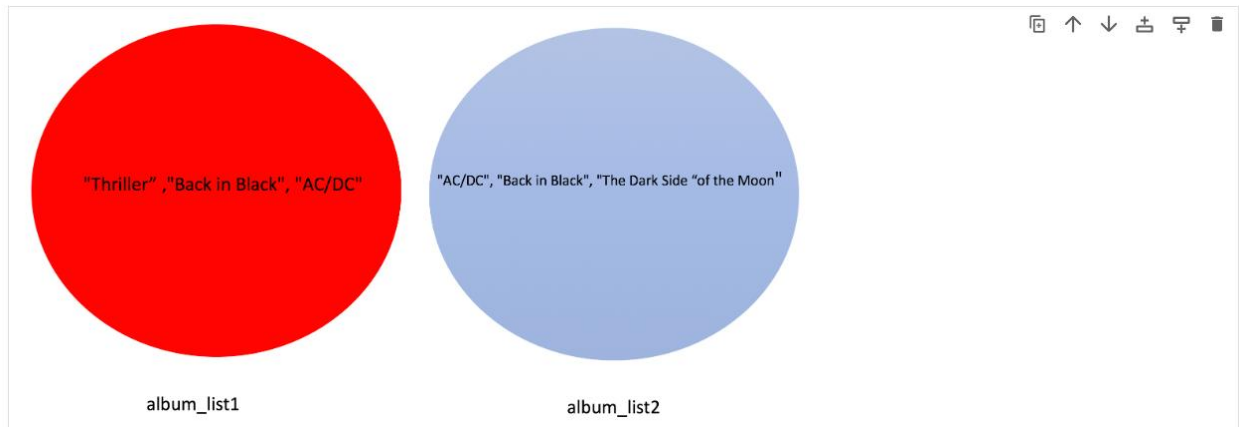
```
True
```

Code:

```
album_set1 = set(["Thriller", 'AC/DC', 'Back in Black'])
```

```
album_set2 = set(["AC/DC", "Back in Black", "The Dark Side of the Moon"])
```

output:



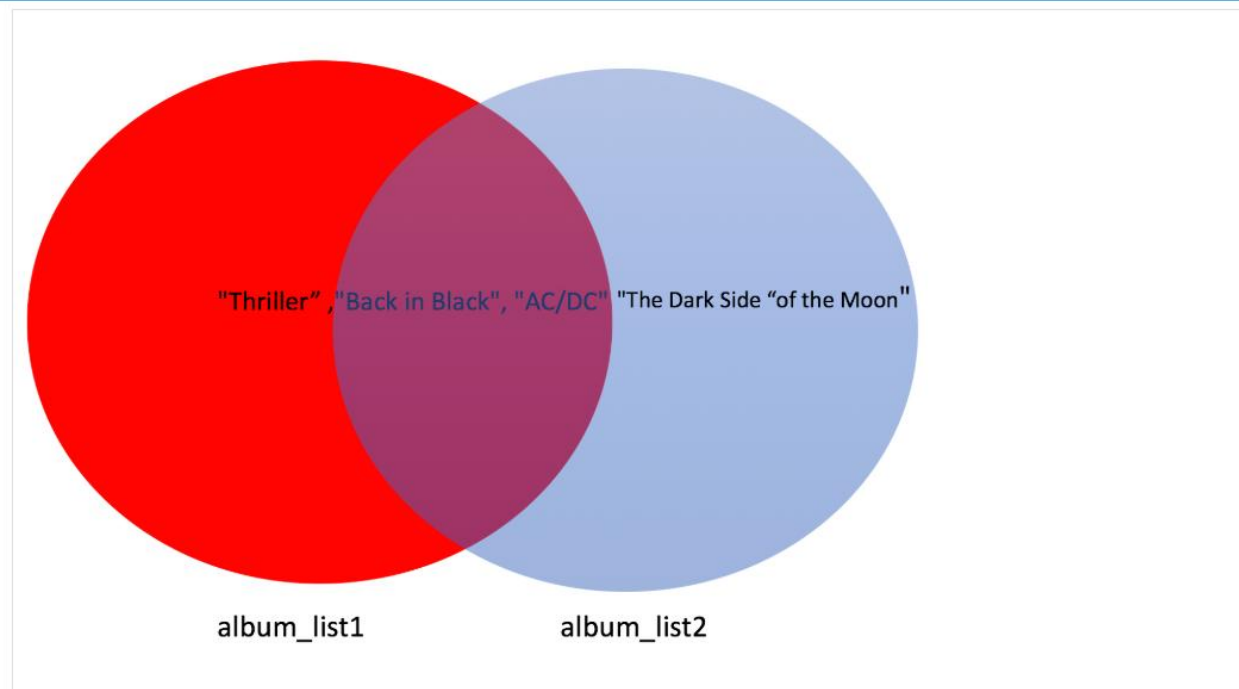
Code:

```
album_set1, album_set2
```

output:

```
{'AC/DC', 'Back in Black', 'Thriller'},  
{'AC/DC', 'Back in Black', 'The Dark Side of the Moon'}}
```

As both sets contain **AC/DC** and **Back in Black** we represent these common elements with the intersection of two circles.



Code:

```
intersection = album_set1 & album_set2  
intersection
```

output:

```
{'AC/DC', 'Back in Black'}
```

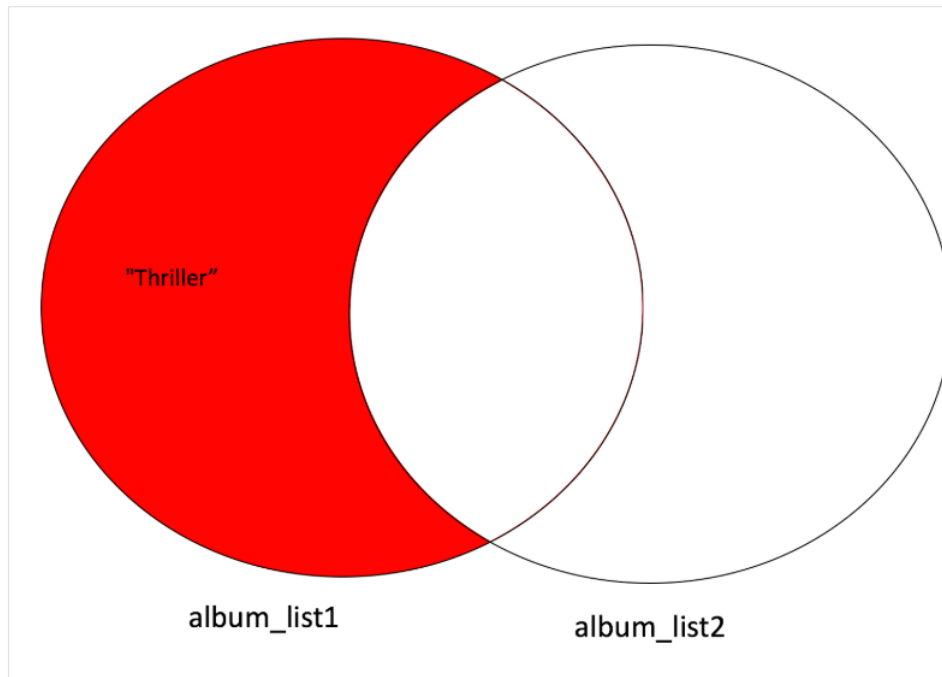
Code:

```
album_set1.difference(album_set2)
```

output:

```
{'Thriller'}
```

You only need to consider elements in `album_set1`; all the elements in `album_set2`, including the intersection, are not included.



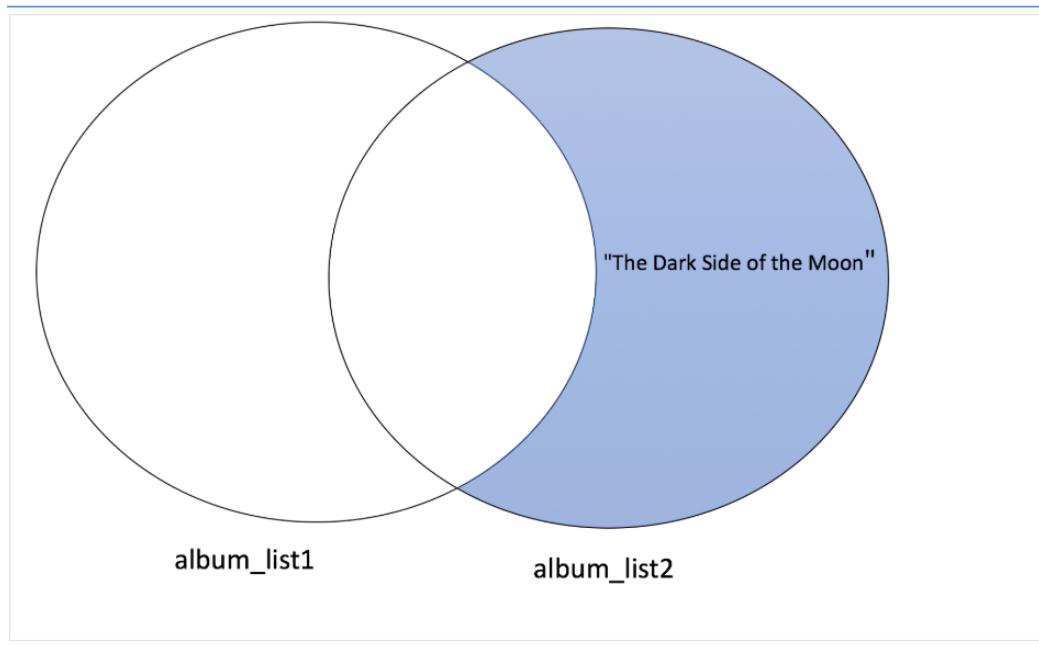
The elements in `album_set2` but not in `album_set1` is given by:

Code:

```
album_set2.difference(album_set1)
```

output:

```
{'The Dark Side of the Moon'}
```



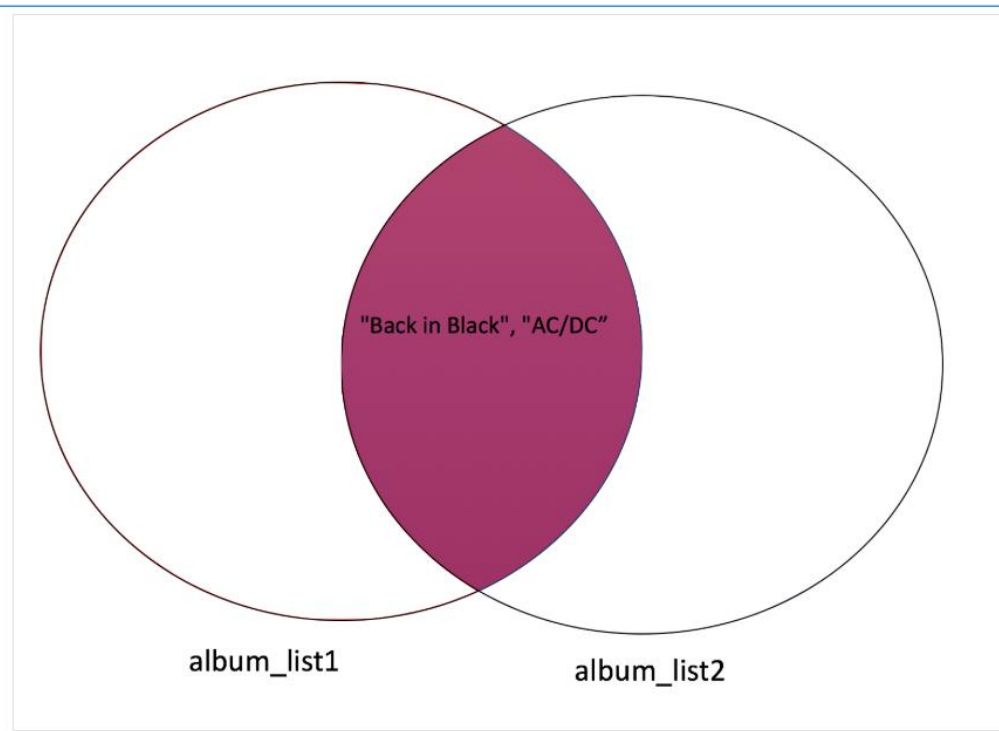
You can also find the intersection of `album_list1` and `album_list2`, using the `intersection` method:

Code:

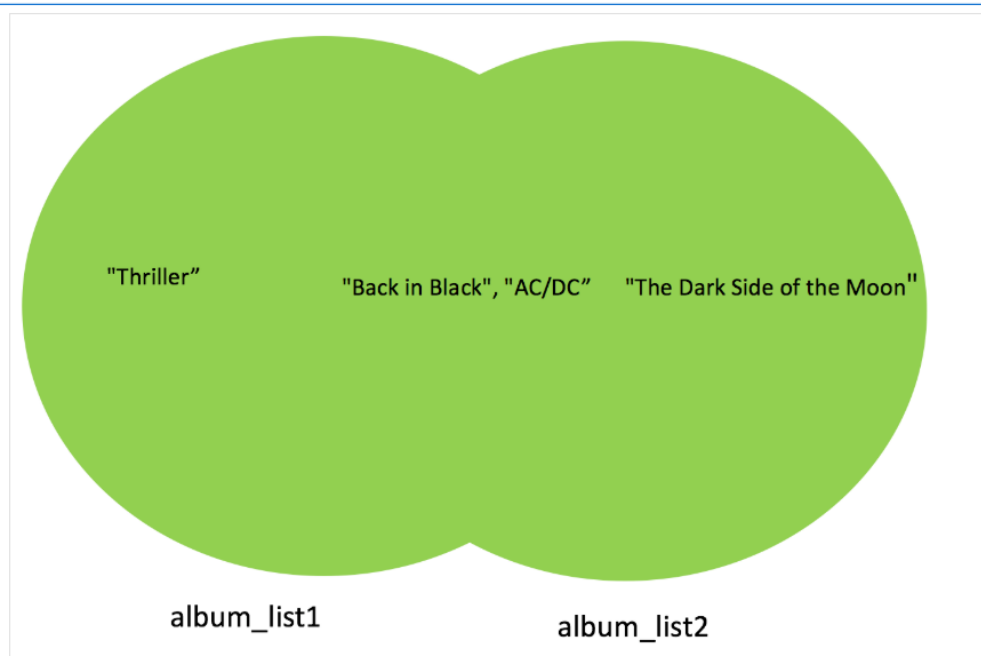
```
album_set1.intersection(album_set2)
```

output:

```
{'AC/DC', 'Back in Black'}
```



The union corresponds to all the elements in both sets, which is represented by coloring both circles:



Code:

```
album_set1.union(album_set2)
```

output:


```
{'AC/DC', 'Back in Black', 'The Dark Side of the Moon', 'Thriller'}
```

Code:

```
set(album_set1).issuperset(album_set2)
```

output:

```
False
```

Code:

```
set(album_set2).issubset(album_set1)
```

output:

```
False
```

Code:

```
set({'Back in Black', 'AC/DC'}).issubset(album_set1)
```

output:

```
True
```

Code:

```
album_set1.issuperset({'Back in Black', 'AC/DC'})
```

output:

```
True
```

Quiz on Sets

Code:

```
album_list = ['rap', 'house', 'electronic music', 'rap']
```

```
album_set = set(album_list)
```

```
album_set
```

output:

```
set(['rap','house','electronic music','rap'])
```

code:

```
A = [1, 2, 2, 1]
```

```
B = set([1, 2, 2, 1])
```

```
print("sum(A) =", sum(A))
```

```
print("sum(B) =", sum(B))
```

```
print("sum(A) == sum(B) =", sum(A) == sum(B))
```

output:

```
sum(A) = 6
```

```
sum(B) = 3
```

```
sum(A) == sum(B) = False
```

code:

```
album_set1 = set(["Thriller", "AC/DC", "Back in Black"])
```

```
album_set2 = set(["AC/DC", "Back in Black", "The Dark Side of the Moon"])
```

```
album_set3 = album_set1.union(album_set2)
```

```
print("album_set3 =", album_set3)
```

output:

```
album_set3 = {'Thriller', 'AC/DC', 'Back in Black', 'The Dark Side of the Moon'}
```

code:

```
album_set1 = set(["Thriller", "AC/DC", "Back in Black"])
```

```
album_set2 = set(["AC/DC", "Back in Black", "The Dark Side of the Moon"])
```

```
album_set3 = album_set1.union(album_set2)
```

```
print("Is album_set1 a subset of album_set3?")
```

```
print(album_set1.issubset(album_set3))
```

output:

Is album_set1 a subset of album_set3?

True

Lab – Dictionaries**Code:**

```
Dict = {"key1": 1, "key2": "2", "key3": [3, 3, 3], "key4": (4, 4, 4), ('key5'): 5, (0, 1): 6}
```

Dict

Output:

```
{'key1': 1,  
'key2': '2',  
'key3': [3, 3, 3],  
'key4': (4, 4, 4),  
'key5': 5,  
(0, 1): 6}
```

Code:

```
Dict["key1"]
```

Output:

1

Code:

```
Dict[(0, 1)]
```

Output:

6

Code:

```
release_year_dict = {"Thriller": "1982", "Back in Black": "1980", \
                     "The Dark Side of the Moon": "1973", "The Bodyguard": "1992", \
                     "Bat Out of Hell": "1977", "Their Greatest Hits (1971-1975)": "1976", \
                     "Saturday Night Fever": "1977", "Rumours": "1977"}

release_year_dict
```

output:

```
{'Thriller': '1982',
 'Back in Black': '1980',
 'The Dark Side of the Moon': '1973',
 'The Bodyguard': '1992',
 'Bat Out of Hell': '1977',
 'Their Greatest Hits (1971-1975)': '1976',
 'Saturday Night Fever': '1977',
 'Rumours': '1977'}
```

Keys

It is helpful to visualize the dictionary as a table, as in the following image. The first column represents the keys, the second column represents the values.

Key	
"Thriller"	"1982"
"Back in Black	"1980"
"The Dark Side of the Moon"	"1973"
"The Bodyguard"	"1992"
"Bat Out of Hell"	"1977"
"Their Greatest..."	"1976"
Saturday Night Fever	"1977"
"Rumours"	"1977"
Value	

Code:

```
release_year_dict['Thriller']
```

output:

```
'1982'
```

This corresponds to:

"Thriller"	"1982"
"Back in Black"	"1980"
"The Dark Side of the Moon"	"1973"
"The Bodyguard"	"1992"
"Bat Out of Hell"	"1977"
"Their Greatest..."	"1976"
"Saturday Night Fever"	"1977"
"Rumours"	"1977"

Code:

```
release_year_dict['The Bodyguard']
```

output:

```
'1992'
```

"Thriller"	"1982"
"Back in Black"	"1980"
"The Dark Side of the Moon"	"1973"
"The Bodyguard"	"1992"
"Bat Out of Hell"	"1977"
"Their Greatest..."	"1976"
"Saturday Night Fever"	"1977"
"Rumours"	"1977"

Now let us retrieve the keys of the dictionary using the method `keys()` :

Code:

```
release_year_dict.keys()
```

output:

```
dict_keys(['Thriller', 'Back in Black', 'The Dark Side of the Moon', 'The Bodyguard', 'Bat Out of Hell', 'Their Greatest Hits (1971-1975)', 'Saturday Night Fever', 'Rumours'])
```

code:

```
release_year_dict.values()
```

output:

```
dict_values(['1982', '1980', '1973', '1992', '1977', '1976', '1977', '1977'])
```

code:

```
release_year_dict['Graduation'] = '2007'
```

release_year_dict

output:

```
{ 'Thriller': '1982',  
  
  'Back in Black': '1980',  
  
  'The Dark Side of the Moon': '1973',  
  
  'The Bodyguard': '1992',  
  
  'Bat Out of Hell': '1977',  
  
  'Their Greatest Hits (1971-1975)': '1976',  
  
  'Saturday Night Fever': '1977',  
  
  'Rumours': '1977',  
  
  'Graduation': '2007'}
```

Code:

```
del(release_year_dict['Thriller'])  
  
del(release_year_dict['Graduation'])  
  
release_year_dict
```

output:

```
{'Back in Black': '1980',  
  
  'The Dark Side of the Moon': '1973',  
  
  'The Bodyguard': '1992',  
  
  'Bat Out of Hell': '1977',  
  
  'Their Greatest Hits (1971-1975)': '1976',  
  
  'Saturday Night Fever': '1977',  
  
  'Rumours': '1977'}
```

Code:

```
'The Bodyguard' in release_year_dict
```

Output:

```
True
```

Quiz on Dictionaries

Code:

```
album_sales_dict = {  
    "The Bodyguard": 50,  
    "Back in Black": 50,  
    "Thriller": 65  
}  
  
print(album_sales_dict)
```

output:

```
{'The Bodyguard': 50, 'Back in Black': 50, 'Thriller': 65}
```

Code:

```
album_sales_dict = {  
    "The Bodyguard": 50,  
    "Back in Black": 50,  
    "Thriller": 65  
}  
  
print(album_sales_dict["Thriller"])
```

output:

```
65
```

Code:


```
album_sales_dict = {  
    "The Bodyguard": 50,  
    "Back in Black": 50,  
    "Thriller": 65  
}  
  
print(album_sales_dict.keys())
```

output:

```
dict_keys(['The Bodyguard', 'Back in Black', 'Thriller'])
```

code:

```
album_sales_dict = {  
    "The Bodyguard": 50,  
    "Back in Black": 50,  
    "Thriller": 65  
}  
  
print(album_sales_dict.values())
```

output:

```
dict_values([50, 50, 65])
```

code:

```
soundtrack_dic = {  
    "The Bodyguard": "1992",  
    "Saturday Night Fever": "1977"  
}  
  
print(soundtrack_dic.keys())
```

output:

```
dict_keys(['The Bodyguard', 'Saturday Night Fever'])
```

code:

```
soundtrack_dic = {  
    "The Bodyguard": "1992",  
    "Saturday Night Fever": "1977"  
}  
  
print(soundtrack_dic.values())
```

output:

```
dict_values(['1992', '1977'])
```